CBNU

MetaInf

대규모 모델 추론 가속

Meta-Learning for Speeding Up Large Model Inference in Decentralized Environments

논문 세미나 | COLM 2025 | arXiv v1 · 2025.08.08

Yipeng Du, Zihao Wang et al. · Nesa Research

발표 흐름

핵심 질문

지금의 환경에서
어떤 가속 조합이
가장 빠른가 ?

1. 문제와 배경 · 03–11

- LLM 추론과 세 가지 가속 기법
- 배치 · 모델 ·GPU 에 따라 바뀌는 최적 조합

2. MetaInf 방법 · 12–19

- 성능 데이터 , LLM 임베딩 , XGBoost
- 오프라인 학습과 예산 내 온라인 선택

3. 실험과 해석 · 20–30

- 선택 정확도 , 순위 , zero-shot, ablation
- 원문 지표의 주의점과 세미나 토론

Du et al., arXiv:2508.09194v1 (COLM 2025 표기)

논문 한눈에 보기 : 가속 기법의 "선택"을 학습

- 연구 문제 : 모델 · 입력 · 하드웨어가 달라지면 최적 가속 조합도 달라짐
- 핵심 아이디어 : 과거 실험 결과로 새 환경의 실행 시간을 예측
- 제안 방법 : LLM 기반 의미 임베딩 + SVD + XGBoost 예측기
- 보고 결과 : 선택 정확도 89.8%, F1 0.85, Accel. 지표 1.55
- 기여 범위 : 기존 가속 기법의 조합을 고르는 메타 스케줄러

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · Abstract, §1, Table 4

왜 탈중앙화 환경에서 선택 문제가 어려운가 ?

- 하드웨어 이질성 : 노드마다 GPU 연산 성능 · 메모리 용량이 다름
- 통신과 부하 변동 : 네트워크 지연 · 대역폭 · 요청 도착 패턴이 달라짐
- 워크로드 다양성 : 입력 길이 · 공통 prefix· 배치 규모가 성능에 영향
- 고정 설정의 한계 : 한 환경에서 빠른 조합이 다른 환경에서는 느릴 수 있음
- 요구사항 : 모든 후보를 매번 실행하지 않고 환경에 맞는 조합 선택

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · §1–2 (배경); §3.3 (선택 문제)

LLM 추론의 두 단계 : Prefill 과 Decode

입력 프롬프트

↓

Prefill

입력 토큰 처리 · KV cache 생성

↓

Decode

이전 KV cache 를 활용해

출력 토큰을 순차 생성

- 어떤 병목을 줄이는가 ?

• Prefill: 입력 전체를 처리하는 단계
긴 프롬프트의 초기 처리 부담

• Decode: 출력 토큰을 반복 생성
요청 길이가 달라 GPU 활용이 불균일

• Prefix caching → 입력 계산 재사용
Chunked prefill → 입력 처리 분할
Continuous batching → 요청 배치 갱신

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · §2, §4.1 · 도식은 원리 설명용 재구성

Continuous Batching: 완료된 자리를 즉시 채우기

Decode 반복마다 배치 구성 갱신



완료 요청 제거 → 대기 요청 삽입

- **Iteration-level scheduling**

- 작동 : 생성 반복 단위로 요청 추가 · 제거
  모든 요청이 끝날 때까지 기다리지 않음

- 기대 효과 : 요청 길이가 달라도
  배치를 유지하며 GPU 활용도 향상

- 확인할 요소 : 도착률 · 동시 요청 수
  스케줄링 부담과 지연 목표

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · §2, §4.1 · 요청 배치 도식은 개념 설명용

Prefix Caching: 같은 입력 prefix 의 계산 재사용

공통 system prompt / 대화 prefix



공통 prefix: cache 에서 재사용

새 입력 : 필요한 부분만 계산

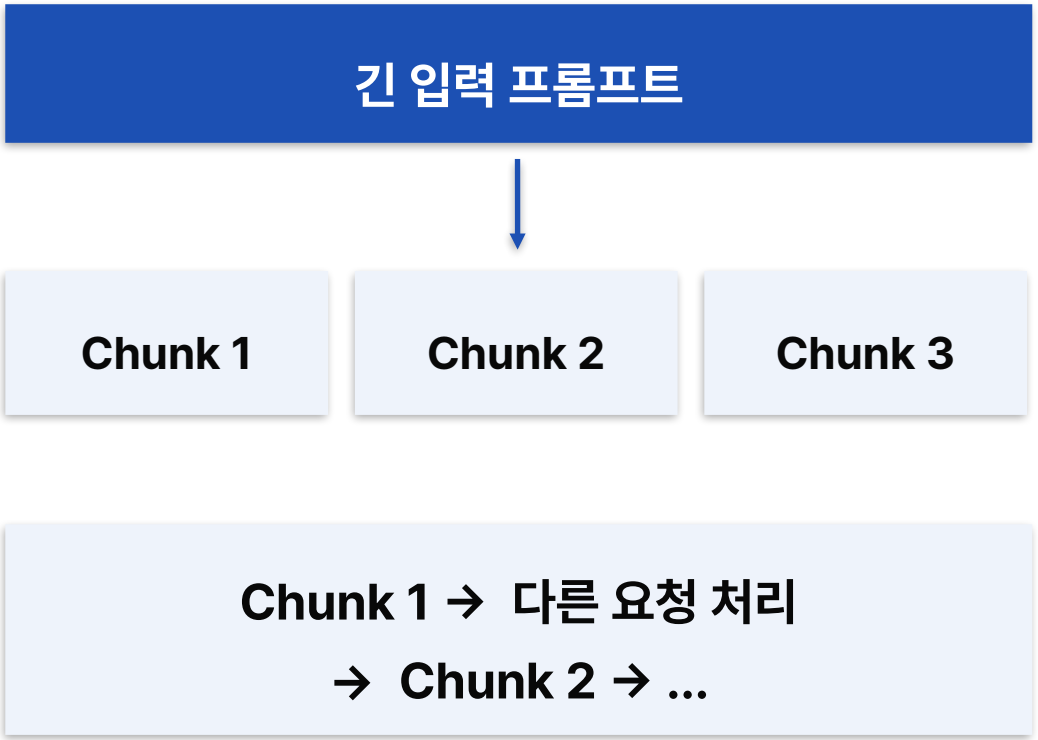
- **캐시 적중과 메모리가 핵심**

- **작동 : 일치하는 prefix 의 KV 값을 재사용해 중복 입력 처리 감소**
- **기대 효과 : 공유 prefix 가 많은 워크로드에서 계산 절약 가능**
- **비용 : cache 메모리와 관리 부담 모델 · 워크로드에 따라 이득이 바뀜**

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · §2, §4.1 · KV cache 도식은 원리 설명용

Chunked Prefill: 긴 입력 처리를 작은 조각으로

긴 프롬프트를 여러 chunk 로 분할



- 분할 크기와 실행 비용의 균형

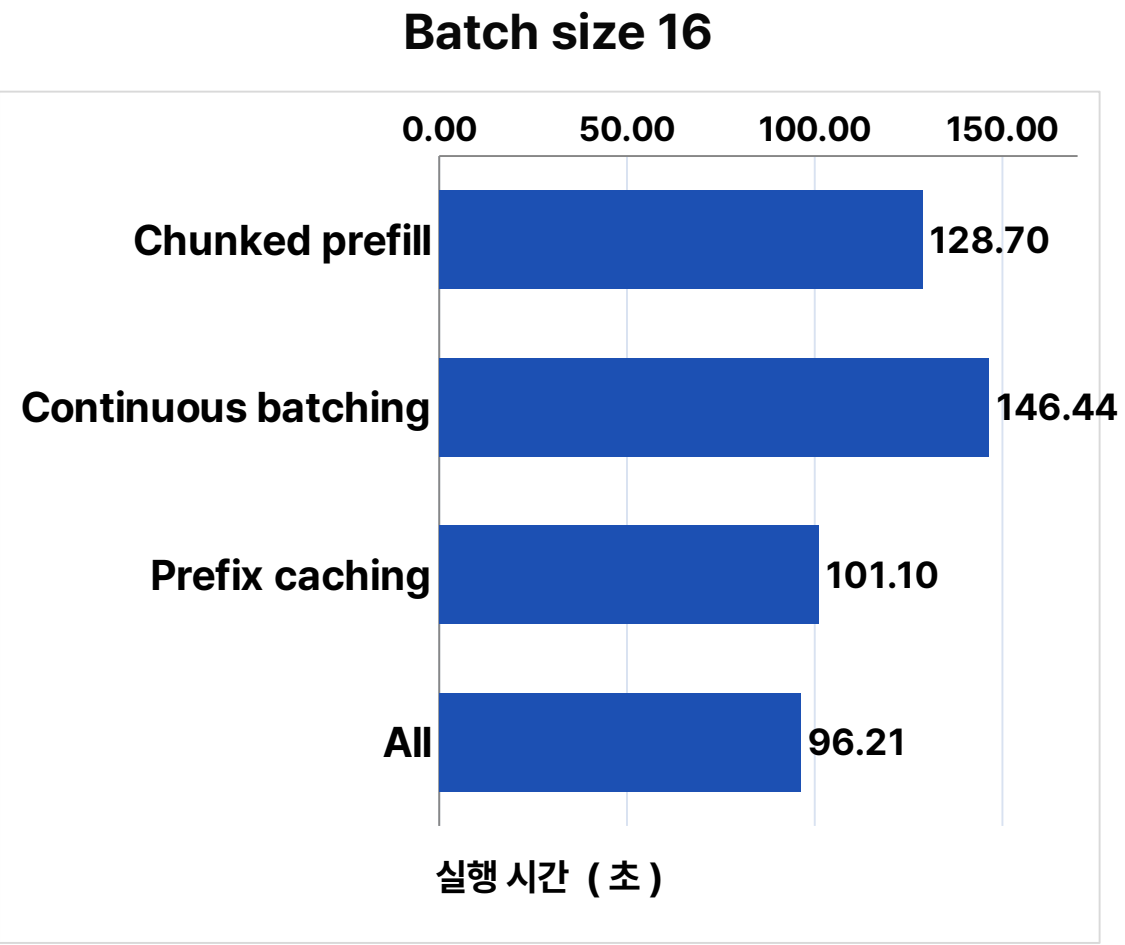
- 작동 : prefill 을 작은 단위로 나눠
다른 요청 작업과 함께 스케줄링

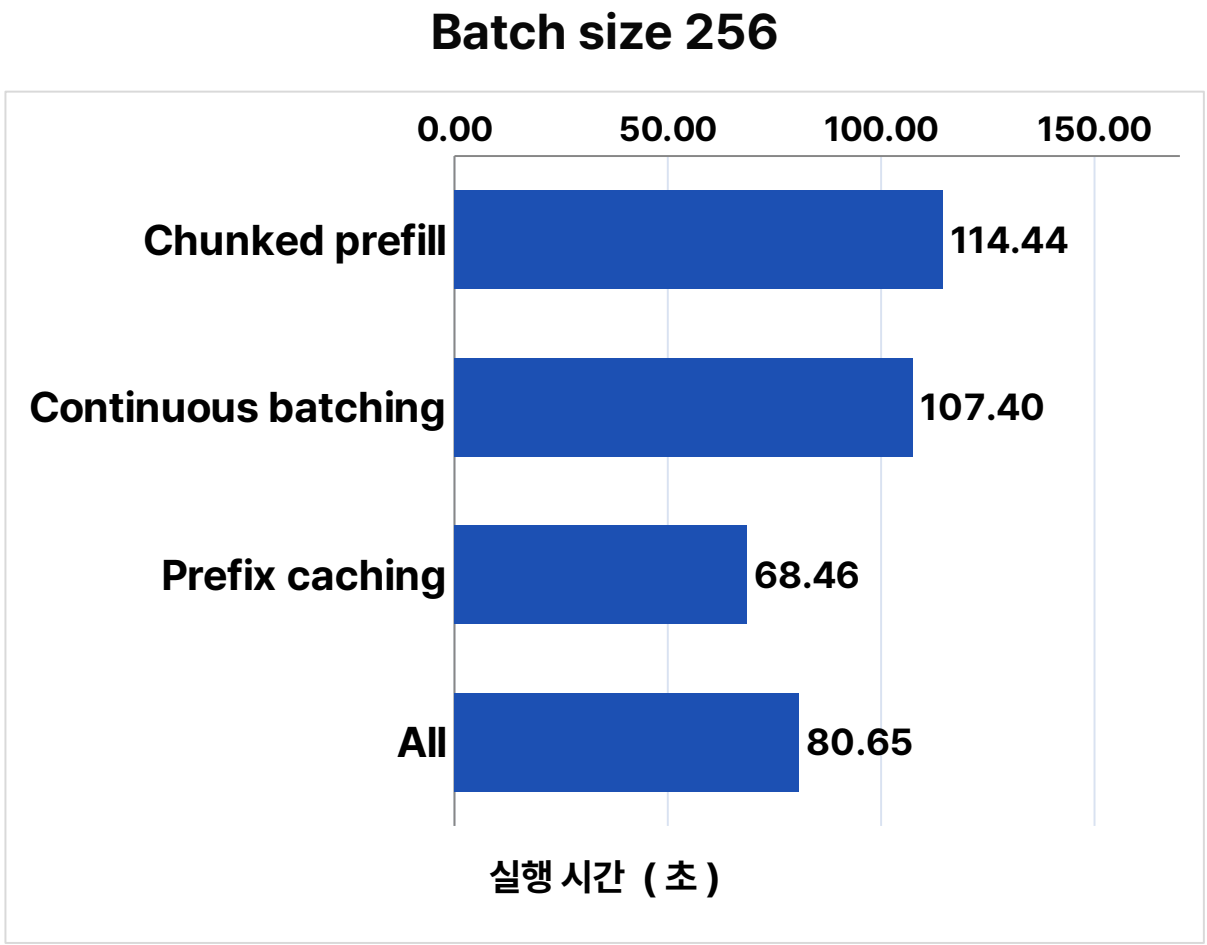
- 기대 효과 : 긴 입력 처리의
한 번에 집중되는 부담을 완화

- 비용 : 추가 스케줄링 · 통신 부담
작은 chunk 가 항상 최선은 아님

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · §1–2, §4.1 · 도식은 개념 설명용

배치 크기가 바뀌면 최적 조합도 바뀐다





미가속 (None): 1435.27 초 / 1424.99 초 · 그래프에서 제외하여 가속 기법 간 차이를 표시

- **BS=16 은 All, BS=256 은 Prefix caching 이 가장 빠름**

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · Table 2; 그래프는 원문 수치 재작성

같은 GPU 에서도 모델에 따라 효과가 달라진다

모델	Chunked prefill	Prefix caching	All
Baichuan2-7B-Chat	+3.82%	+37.63%	+7.96%
Qwen2.5-7B-Instruct-1M	+4.15%	−10.66%	−7.20%
Phi-2	−0.90%	−56.79%	−11.39%
Meta-Llama-3.1-8B-Instruct	+5.40%	−4.03%	+0.33%

- **Prefix caching 의 효과는 모델별로 반대 방향**

- NVIDIA L4, BS=1024, continuous batching 은 기본 활성화
- 표의 ±% 는 원문 표기 · 양수는 성능 향상 , 음수는 성능 저하로 설명

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · Table 1, §1 · 변화율의 세부 산식은 원문 미명시

GPU 를 두 배로 늘려도 개선 폭은 제한적

가속 조합	4 GPU 시간 (초)	8 GPU 시간 (초)
Chunked prefill	120.89 (5.43)	117.21 (5.15)
Continuous batching	122.73 (12.75)	118.19 (13.87)
Prefix caching	79.18 (11.86)	76.33 (11.92)
All	85.07 (6.04)	82.90 (5.96)

- 하드웨어 증설과 가속 기법 선택을 함께 고려

- 4→8 GPU 시간 감소 : 약 2.6~3.7% (표 수치로 계산)
- 괄호 값은 원문 그대로 · 분산 / 표준편차 중 어떤 정의인지 미명시

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · Table 3, §3.1–3.2 · 본문은 처리량 개선을 약 5% 로 서술

MetaInf 전체 구조 : 과거 성능으로 새 조합을 예측

① Offline: 과거 환경에서 성능을 학습

과거 실행 기록 → 설명 → 임베딩 → SVD → XGBoost 성능 예측기 f → 성능 매핑 학습

② Online: 새 환경의 가속 조합을 선택

새 입력 · 모델 ·GPU → 동일한 임베딩 ·SVD 변환 → 후보별 시간 · 비용 예측 → 예산 내 최적 조합 실행

새 환경의 모든 후보를 실제로 실행하는 과정을 예측으로 대체

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · Figure 2, §3.3–3.5 · 편집 가능한 도식으로 재구성

문제 정의 : 실행 성능과 배포 예산을 함께 고려

$$M^* = \text{argmin } t_pred(j, k) \quad \text{subject to } r_k \times t_pred(j, k) \le b$$

- M_j: 가속 조합 / H_k: 실행 GPU 구성 / b: 허용 비용
- t_pred(j, k): 새 작업에서 후보 조합의 예측 실행 시간
  r_k: GPU 구성의 단위 시간당 비용 → C_pred = r_k × t_pred
- 원문 Eq. (2) 는 성능 P 의 argmax 로 표현
  여기서는 시간을 목표로 두어 argmin 형태로 설명

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · §3.3, Eq. (2) · 시간 최소화 형태로 풀어 쓴 설명식

메타학습 데이터 : 작업 × 가속 조합 × 하드웨어

성능 텐서 P: n × m × h 배열

기록 축	저장 내용
작업 D_i	입력 · 데이터 특성
조합 M_j	가속 기법 ON/OFF
하드웨어 H_k	GPU / 실행 구성
관측값 P_i,j,k	시간 등 성능 지표

하나의 학습 샘플

(환경 표현 , 실제 성능)

- **새 작업에 이전 경험을 전이**

- **과거 : 여러 작업 · 가속 조합 ·GPU 에서 실제 실행 성능을 측정**
- **학습 : 환경 설명을 입력으로 받아 관측 실행 시간을 맞추도록 회귀**
- **새 작업 : 유사한 과거 환경의 성능 관계를 활용해 후보를 비교**

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · §3.3–3.4, Eq. (1) · 표는 데이터 구조 설명용

LLM 임베딩 : 환경 설명을 의미 있는 벡터로

작업 : 입력 규모 · 분포 · 복잡도

모델 / 기법 : 구조 · 연산 · 가속 특성

GPU: 아키텍처 · 메모리 · 제약

사전학습 LLM → Dense embedding

Truncated SVD → 저차원 표현

- One-hot 과 무엇이 다른가 ?

- One-hot: 범주의 식별자를 구분
범주 사이 의미적 관계를 직접 표현하지 않음

- LLM 표현 : 설명 문장의 맥락 활용
구조 · 자원 제약의 유사성을 담도록 설계

- SVD: 벡터 차원을 줄여 효율 확보
온라인에도 학습 시 변환을 동일 적용

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · §3.4, §4.2 · 예시는 설명용 재작성

성능 예측기 f: 임베딩을 XGBoost 회귀로 연결

$$P_{pred}(i, j, k) = f(E_{data}(i), E_{method}(j), E_{hardware}(k))$$

- 환경 임베딩을 묶어 회귀 예측기에 입력
  XGBoost 로 후보별 실행 시간 등 성능을 예측
- 논문이 제시한 선택 이유 : 이질적 입력에 대한 강건성
  특성 중요도를 통해 예측 관계를 해석할 수 있음
- Ablation 의 구체적 입력 축은 5 개
  모델 이름 , GPU 아키텍처 , 세 가속 기법의 boolean flag

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · Eq. (1), §3.4, §4.6

Offline 학습 : 성능 데이터와 예측기를 미리 준비

1. 성능 측정	여러 작업 · 모델 ·GPU 에서 후보 조합을 실행 실제 실행 시간을 성능 데이터로 저장
2. 환경 표현	작업 · 모델 / 기법 · 하드웨어 설명 → LLM 임베딩 SVD 로 차원을 축소하고 입력 특성을 구성
3. 예측기 학습	입력 표현과 관측 시간을 묶어 XGBoost 회귀 학습 새 환경의 후보 성능을 비교할 예측기 f 확보
4. 온라인 준비	임베딩 방식 ·SVD 변환 · 예측기를 함께 유지 온라인 후보도 같은 표현 공간에서 평가

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · §3.4, Appendix A.1 Algorithm 1

Online 선택 : 실제 후보 실행 없이 조합을 고르기

1. 새 환경 입력

새 작업 , 사용할 모델과 GPU, 허용 예산 b 입력
오프라인과 동일한 임베딩 ·SVD 변환 적용

2. 후보 평가

각 가속 조합에 대해 실행 시간을 예측
시간 × GPU 단가로 예상 배포 비용 산출

3. 예산 필터

예상 비용이 b 이하인 후보만 비교
시간 목표라면 예상 실행 시간이 가장 짧은 후보 선택

4. 조합 실행

선택한 조합으로 실제 추론 수행
새 환경에서 예측기 재학습은 요구하지 않음

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · §3.5, Appendix A.1 Algorithm 2

예산 제약의 의미 : 가장 빠른 후보가 제외될 수 있음

가상 후보	예측 시간	단가 / 비용	예산 판정
A: GPU 구성 1 + 조합 A	8 초	1 원 / 초 → 8 원	가능
B: GPU 구성 2 + 조합 B	5 초	3 원 / 초 → 15 원	제외
C: GPU 구성 1 + 조합 C	6 초	1 원 / 초 → 6 원	가능

- 설명용 가상 예시 : 예산 b=10 원 → 후보 C 선택

- B 가 가장 빠르지만 예산을 초과 · 허용 후보 A/C 중 C 가 빠름
- 예측 오차가 있으면 실제 시간 · 비용이 달라질 수 있음

출처 : §3.5 의 제약식에 대한 자체 설명 · 모든 숫자는 가상값 , 실험 결과 아님

실험 설정 : 무엇을 대상으로 선택 성능을 검증했나 ?

- 모델 : Baichuan2-7B-Chat, Qwen2.5-7B-Instruct-1M,
Phi-2, Meta-Llama-3.1-8B-Instruct

- 워크로드 : reasoning-v1-20m + ShareGPT
구조화된 추론 입력과 실제 대화 입력을 함께 사용

- 하드웨어 : NVIDIA T4 16GB, L4 24GB, A100 40GB

- 선택 대상 : Prefix caching / Chunked prefill /
Continuous batching 을 사용하는 가속 조합

- 평가 : 미학습 입력 1,000 개에서 모든 후보를 실행해 실제 최적 조합 확인

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · §4.1, Appendix A.2 · 2048/float16 은 §3.1 동기 실험 조건

평가 지표 : “ 맞게 선택”과 “실제로 빠름”은 다르다

- **Selection accuracy: 예측 선택이 실제 최단 시간 조합과 일치한 비율**

- **F1: 조합 선택의 precision/recall 을 요약**

클래스별 집계 방식 (macro/weighted 등) 은 원문 미명시

- **Mean rank: 예측 시간 순위에서 실제 최적 조합의 평균 위치**

작을수록 좋음 · 최선은 1 위

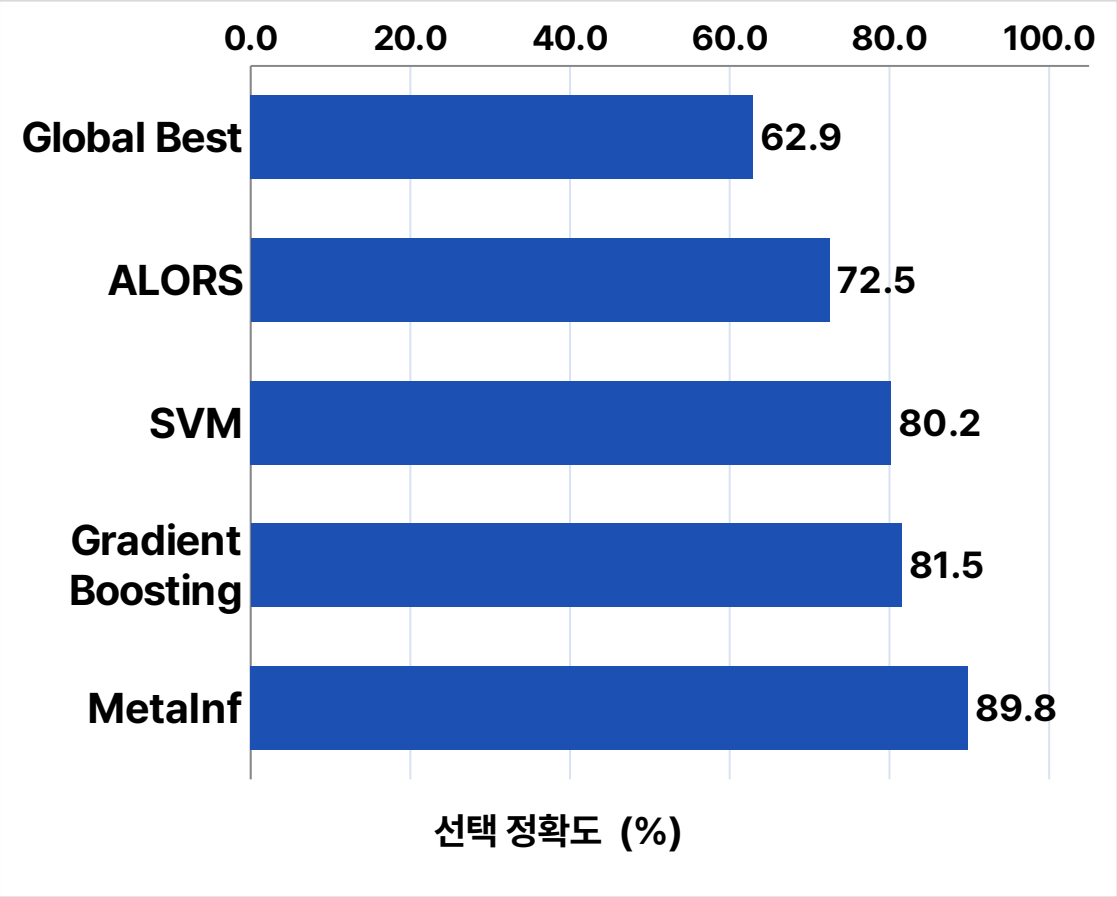
- **Accel.: 원문 Table 4 에서 MetaInf 는 1.55 로 보고**

§4.3 의 비율 설명과 “클수록 좋음” 해석 사이에 불일치

- **발표 시 1.55 를 특정 기준 대비 “정확히 55% 빨라짐”으로 단정하지 않음**

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · Table 4, §4.3, §4.5, Appendix A.2

선택 정확도 : MetaInf 89.8%, 가장 높은 보고값



- 핵심 비교

- MetaInf: 89.8%
Gradient Boosting: 81.5%

- 절대 차이 +8.3%p
환경을 보고 조합을 고르는 능력 개선

- 정확도는 조합 선택의 일치율
LLM 답변의 정답률을 뜻하지 않음

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · Table 4 · 그래프는 원문 수치 재작성

종합 결과 : 정확도 ·F1· 보고 가속 지표 모두 우수

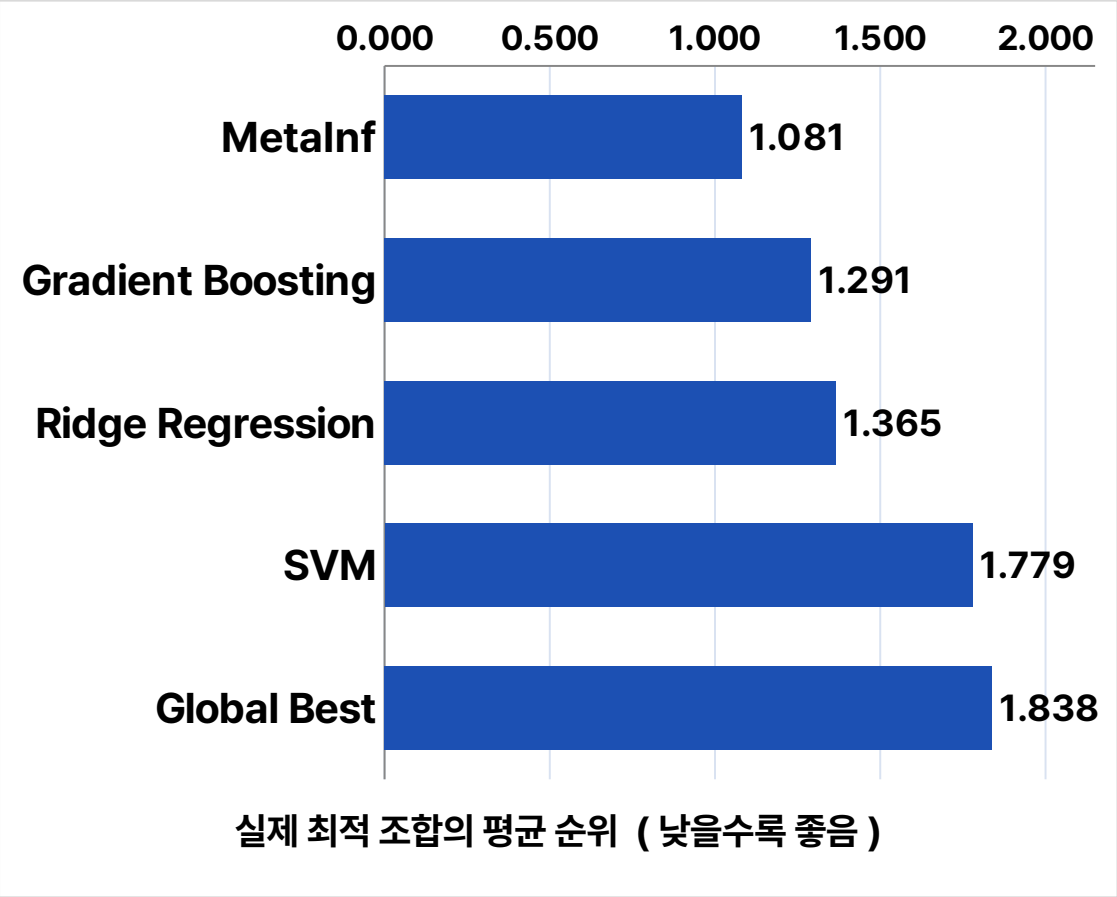
선택 방법	Accuracy / F1	Accel. (원문)
Global Best	0.629 / 0.63	1.16
ALORS	0.725 / 0.71	1.20
SVM	0.802 / 0.79	1.28
Gradient Boosting	0.815 / 0.78	1.30
MetaInf	0.898 / 0.85	1.55

- 전체 비교 11 개 방법 중 MetaInf 가 최고 보고값

- F1 0.85 · 다음으로 높은 SVM 의 F1 은 0.79
- Accel. 1.55 는 원문 값 그대로 · 전체 baseline 표는 부록 31 장

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · Table 4 · Accel. 정의 불일치는 21·28 장에서 설명

실제 최적 조합을 예측 순위의 상단에 배치



- 단순 일치율을 보완하는 지표

- 각 환경에서 후보를 예측 시간순 정렬
실제로 가장 빠른 조합의 위치 기록

- 평균 순위가 1 에 가까울수록
실제 최적 조합을 우선 추천

- 잘못 고른 경우의 시간 손실 크기는
이 지표만으로 알 수 없음

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · Figure 3, §4.5 · 원그림 라벨 수치 재작성

Zero-shot: 새 모델 ·GPU 조합 6 개 중 5 개에서 개선

미관측 모델 / GPU	MetaInf (초)	CB 기준 (초)	시간 변화
Mistral-7B / A100	179.41	202.90	−11.6%
Mistral-7B / H200	31.47	38.20	−17.6%
Mixtral-8×7B / A100	215.86	241.92	−10.8%
Mixtral-8×7B / H200	39.85	37.42	+6.5%
LLaMA-3.1-70B / A100	895.24	1012.66	−11.6%
LLaMA-3.1-70B / H200	150.52	168.71	−10.8%

- **Mixtral-8×7B / H200 에서는 오히려 6.5% 느려짐**

- CB = Continuous batching · 음수는 시간 감소 , 양수는 시간 증가
- 미관측 환경으로의 전이 가능성 확인 · 모든 환경의 개선을 보장하지 않음

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · Table 5 · A100 은 학습에 등장 , H200 은 미등장으로 기술

Ablation: 환경 설명의 풍부함이 선택 성능에 영향

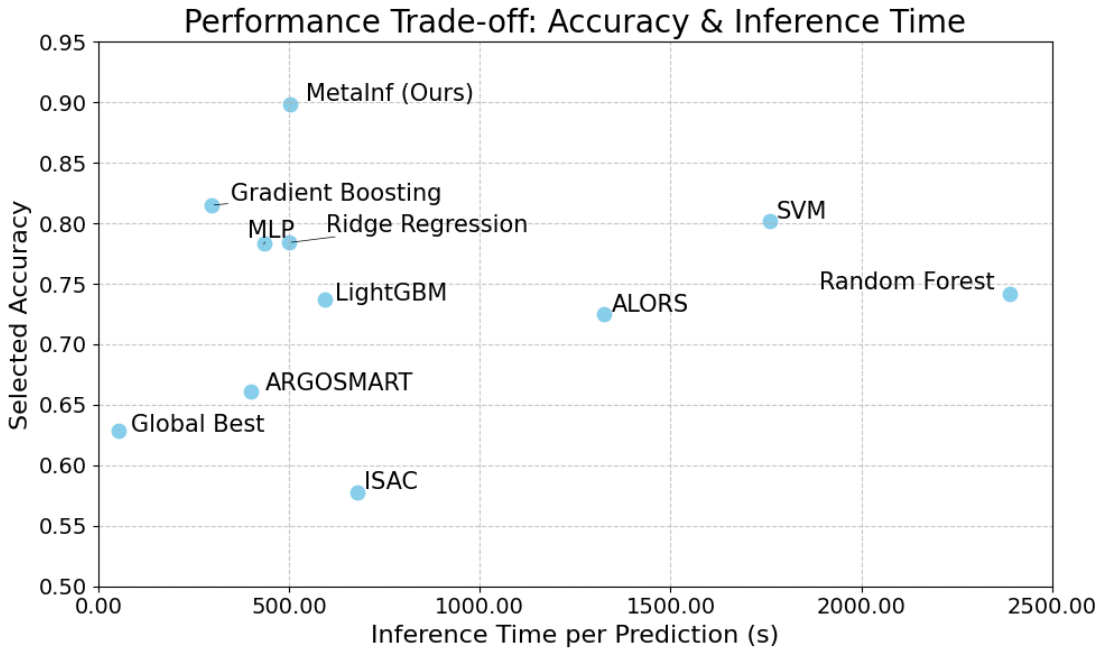
표현 방식	설명 수준	Figure 4 에서의 관찰
One-hot	범주별 식별자 표현	기본 범주 구분 성능
Basic prompt	최소 키워드 · 이름	간단한 의미 정보 활용
Rich prompt	구조 · 연산 · 메모리 맥락	Basic 보다 높은 MetaInf 정확도
CoT prompt	성능 관련 의존관계 설명	MetaInf 의 가장 높은 막대값

- LLM 임베딩에 담는 정보의 질이 중요한 설계 요소

- Figure 4 는 SVD k=256 조건 · MetaInf 가 각 표현 방식에서 우수
- 본문은 k=64~256 비교를 서술 · 차원별 세부 수치 표는 미제공

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · §4.6, Figure 4 · CoT 는 표현 생성 prompt 를 뜻함

선택기의 비용도 중요 : 정확도와 예측 시간의 균형



원문 x 축 : prediction 당 시간 (s)
측정 단위 · 집계 방식 추가 확인 필요

- 그래프 해석의 범위

• MetaInf 는 높은 정확도와 비교적 작은 예측 시간을 함께 보임

• Global Best 는 선택이 단순하지만 선택 정확도는 0.629 로 낮음

• 그림의 시간축은 수백 ~ 수천 "초" 이를 밀리초 단위 실시간 처리의 증거로 해석하기 어려움

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · Figure 5, §4.4 · 왼쪽은 원그림 ; 시간 단위는 원문 표기

비판적 검토 : 결과가 보여주는 범위와 남은 질문

- 지표 정의 : Accel. 비율 방향과 Figure 5 시간축이 불명확

정량적 속도 향상 · 선택 오버헤드 해석에 주의

- 탈중앙화 검증 : GPU 종류 · 수량 비교는 제시되지만

실제 네트워크 변동 · 노드 이탈 실험은 보고되지 않음

- 재현성 : 임베딩 모델의 정확한 버전 · 벡터 추출 방식 ,

학습 데이터 규모 · 분할 및 회귀 설정이 충분히 상세하지 않음

- 예산 보장 : 시간 예측 오차를 고려한 안전 여유나

예산 초과율 · 불확실성 보정 결과는 제시되지 않음

근거 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · §3–4, Appendix A · 한계는 발표자의 원문 검토

결론 : 환경에 맞는 가속 조합을 학습해 선택

- 관찰 : 가속 기법의 효과는 모델 · 배치 ·GPU 조건에 따라 달라짐
- 방법 : LLM 의미 임베딩과 XGBoost 로 후보별 성능 관계를 학습
- 결과 : 선택 정확도 89.8%, 실제 최적 조합의 평균 순위 1.081
- 전이 : 미관측 모델 ·GPU 조합 대부분에서 CB 보다 시간 감소

Mixtral / H200 의 실패 사례는 일반화의 한계도 보여줌

- 의의 : 온라인 전수 탐색을 줄이는 성능 예측 기반 설정 선택

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · §5, Tables 4–5, Figure 3

세미나 토론 : 어떤 검증을 더하면 설득력이 높아질까 ?

- 질문 1: 모델 이름 ·GPU 이름 임베딩만으로 충분한가 ?

입력 길이 ·cache 적중률 · 실제 시스템 부하를 어떻게 반영할까 ?

- 질문 2: 최적 조합을 틀렸을 때 실제 손해는 얼마나 큰가 ?

선택 정확도와 함께 시간 regret·p95/p99 지연이 필요할까 ?

- 질문 3: 예측이 불확실한 새 GPU 에서는 어떻게 대응할까 ?

소수 후보 실측 , 보수적 예산 , 온라인 업데이트를 적용할 수 있을까 ?

- 질문 4: 탈중앙화 시스템의 핵심은 얼마나 검증했는가 ?

네트워크 변동 · 노드 이탈 · 분산 통신 비용을 추가 실험하면 어떨까 ?

발표자 토론 제안 · 논문의 실험 결과나 구현된 기능을 뜻하지 않음

부록 A: 전체 baseline 결과

Heuristic / 비교 회귀 모델

방법	Acc.	F1	Accel.
ISAC	0.578	0.60	1.10
Global Best	0.629	0.63	1.16
ARGOSMART	0.660	0.62	1.17
ALORS	0.725	0.71	1.20
Ridge Regression	0.784	0.73	1.27

학습 기반 모델 / MetaInf

방법	Acc.	F1	Accel.
SVM	0.802	0.79	1.28
MLP	0.783	0.75	1.24
Random Forest	0.742	0.69	1.25
LightGBM	0.737	0.68	1.19
Gradient Boosting	0.815	0.78	1.30
MetaInf	0.898	0.85	1.55

- Accuracy 는 조합 선택 정확도 · Accel. 은 원문 보고값

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · Table 4

부록 B: 알고리즘을 구현 관점에서 요약

Offline meta-training

1 과거 작업 × 후보 × GPU 실행

2 관측 시간 P 저장

3 설명 → LLM 임베딩

4 임베딩 → SVD 변환

5 표현 · 관측 시간으로 f 학습

6 변환과 f 를 함께 저장

Online method selection

1 새 환경을 같은 방식으로 변환

2 각 후보의 시간 t_pred 예측

3 C_pred = GPU 단가 × t_pred

4 C_pred ≤ b 인 후보만 유지

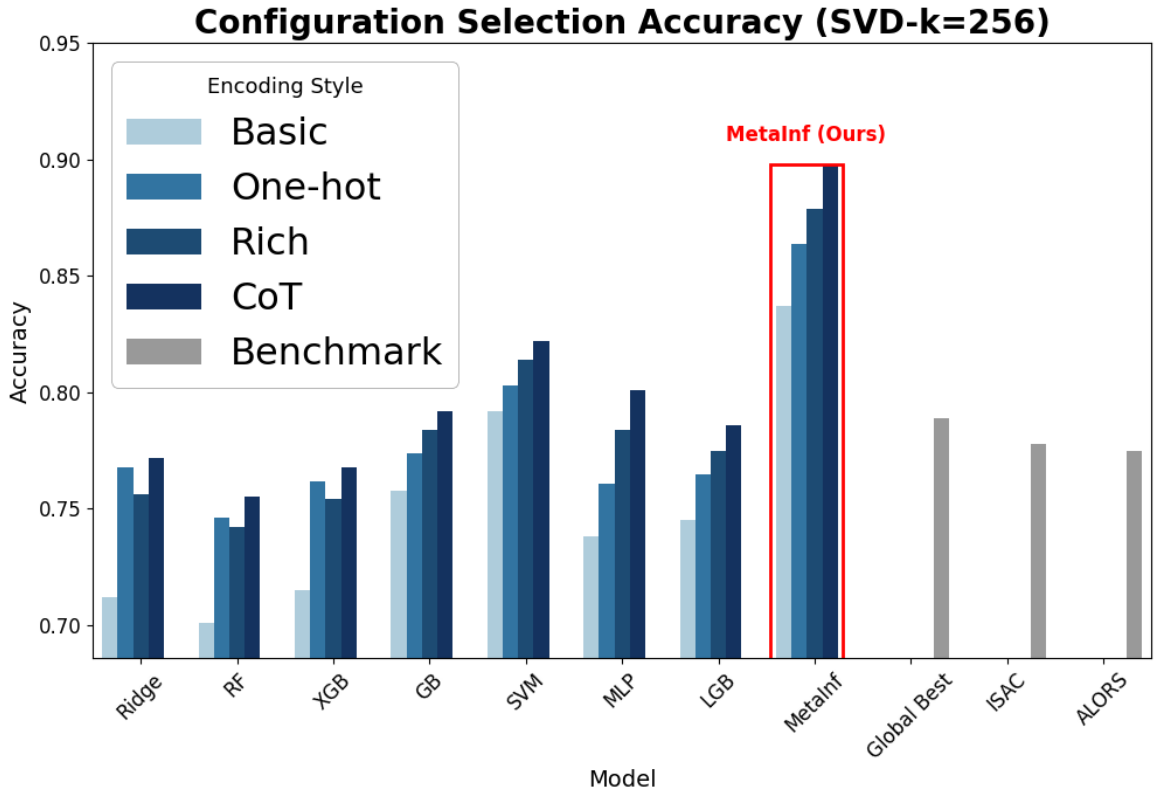
5 예측 시간이 최소인 후보 선택

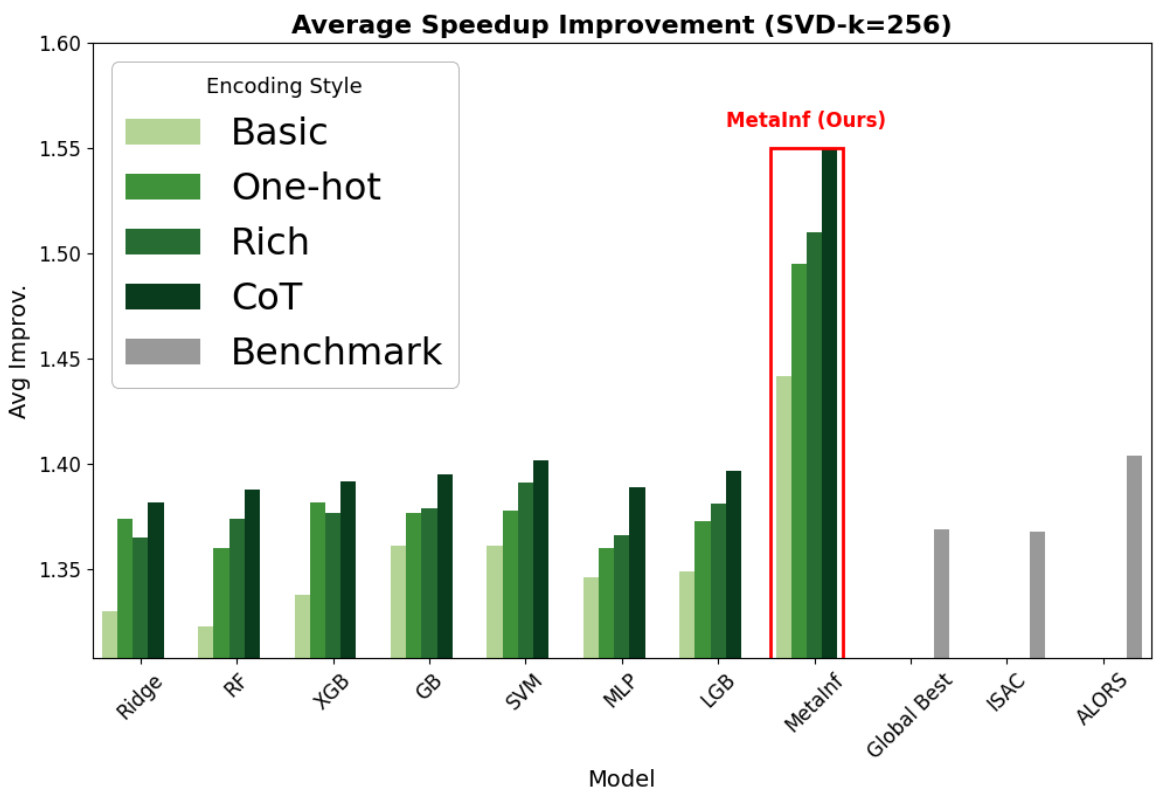
6 선택한 조합으로 추론 실행

- 최적화할 성능 지표의 방향과 비용 단위를 구현 전에 통일

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · Appendix A.1 Algorithms 1–2 · 의사코드 재구성

부록 C: 표현 방식별 ablation 원그림





왼쪽 : 조합 선택 정확도 · 오른쪽 : 평균 speedup improvement

막대 순서 : Basic / One-hot / Rich / CoT · Accel. 해석은 원문 정의 확인 필요

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · Figure 4 (SVD k=256) · 원그림의 값은 임의로 수치화하지 않음

부록 D: 동기 실험과 본 실험의 조건을 구분

- 동기 실험 (§3.1): ShareGPT, 주로 LLaMA-3.1-8B-Instruct

NVIDIA L4, tensor parallelism 4/8 GPU, 최대 출력 2048 tokens, float16

- 추가 모델 비교 (Table 1): L4, BS=1024

Continuous batching 을 기본 활성화한 상태의 성능 변화

- 본 선택 실험 (§4): reasoning-v1-20m + ShareGPT

4 개 모델 , T4/L4/A100, 미학습 입력에서 실제 최적 조합과 비교

- Zero-shot (Table 5): Mistral-7B, Mixtral-8×7B, LLaMA-3.1-70B

A100/H200 의 새로운 모델– GPU 조합을 continuous batching 과 비교

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · §3.1.1, §4.1, Tables 1·5, Appendix A.2

부록 E: 참고 논문과 핵심 용어

- 주 논문 : Du et al., Meta-Learning for Speeding Up Large Model Inference in Decentralized Environments · arXiv:2508.09194v1

- 관련 연구 : Orca (OSDI 2022), PagedAttention (SOSP 2023)
회귀 예측기 : XGBoost (KDD 2016) · 상세 서지는 주 논문 References

- Meta-learning: 과거 환경의 성능 관계를 새 환경 선택에 전이
Zero-shot: 새 조합에서 선택기를 다시 학습하지 않고 사용

- SVD: 임베딩 차원 축소 · KV cache: attention 의 key/value 저장

출처 : Du et al., arXiv:2508.09194v1 (COLM 2025 표기) · References, §3.4–3.5 · 원문 https://arxiv.org/abs/2508.09194